

Module 3

Process Synchronization and Deadlocks

Concurrency

- **Definition:** Concurrency is the execution of the multiple instruction sequences at the same time.
 - Communication among processes
 - Sharing of and competing for resources
 - Synchronization of activities of multiple processes
 - Allocation of processor time to processes

Principles of Concurrency :

- Both interleaved and overlapped processes can be viewed as examples of concurrent processes, they both present the same problems.

The relative speed of execution cannot be predicted. It depends on the following:

- . The activities of other processes
- . The way operating system handles interrupts
- . The scheduling policies of the operating system

Problems & Issues in Concurrency :

Problems in Concurrency :

- Sharing global resources –
- Optimal allocation of resources –
- Locating programming errors –
- Locking the channel –

Issues in Concurrency :

- Non-atomic –
- Race conditions –
- Blocking –
- Starvation –
- Deadlock –

Advantages & Disadvantages of Concurrency

- **Advantages of Concurrency :**

- Running of multiple applications –
 - Better resource utilization –
 - Better average response time –
 - Better performance –

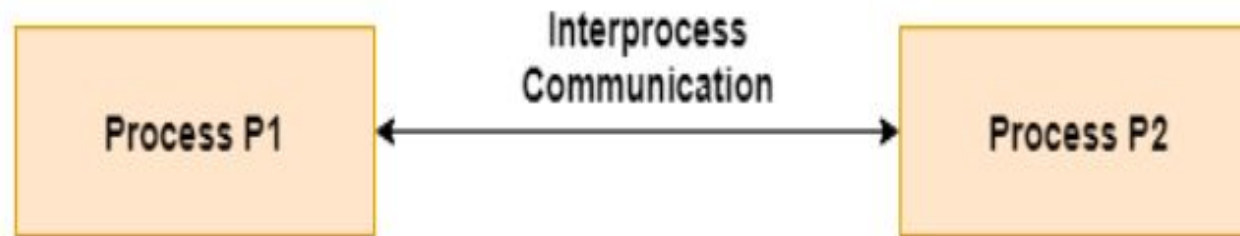
- **Drawbacks of Concurrency :**

- It is required to protect multiple applications from one another.
 - It is required to coordinate multiple applications through additional mechanisms.
 - Additional performance overheads and complexities in operating systems are required for switching among applications.
 - Sometimes running too many applications concurrently leads to severely degraded performance.

Inter Process Communication in OS

•**Definition** : Inter Process Communication (IPC) refers to a mechanism, where the operating systems allow various processes to communicate with each other. This involves synchronizing their actions and managing shared data.

A diagram that illustrates inter process communication is as follows –



•**There are 2 types of process –**

- Independent Processes: The process that does not affect or is affected by the other process while its execution then the process is called Independent Process.
Example The process that does not share any shared variable, database, files, etc.
- Cooperating Processes : The process that affect or is affected by the other process while execution, is called a Cooperating Process. Example The process that share file, variable, database, etc are the Cooperating Process.

•**There are 2 ways by which Inter process communication is achieved –**

- Shared memory
- Message Passing

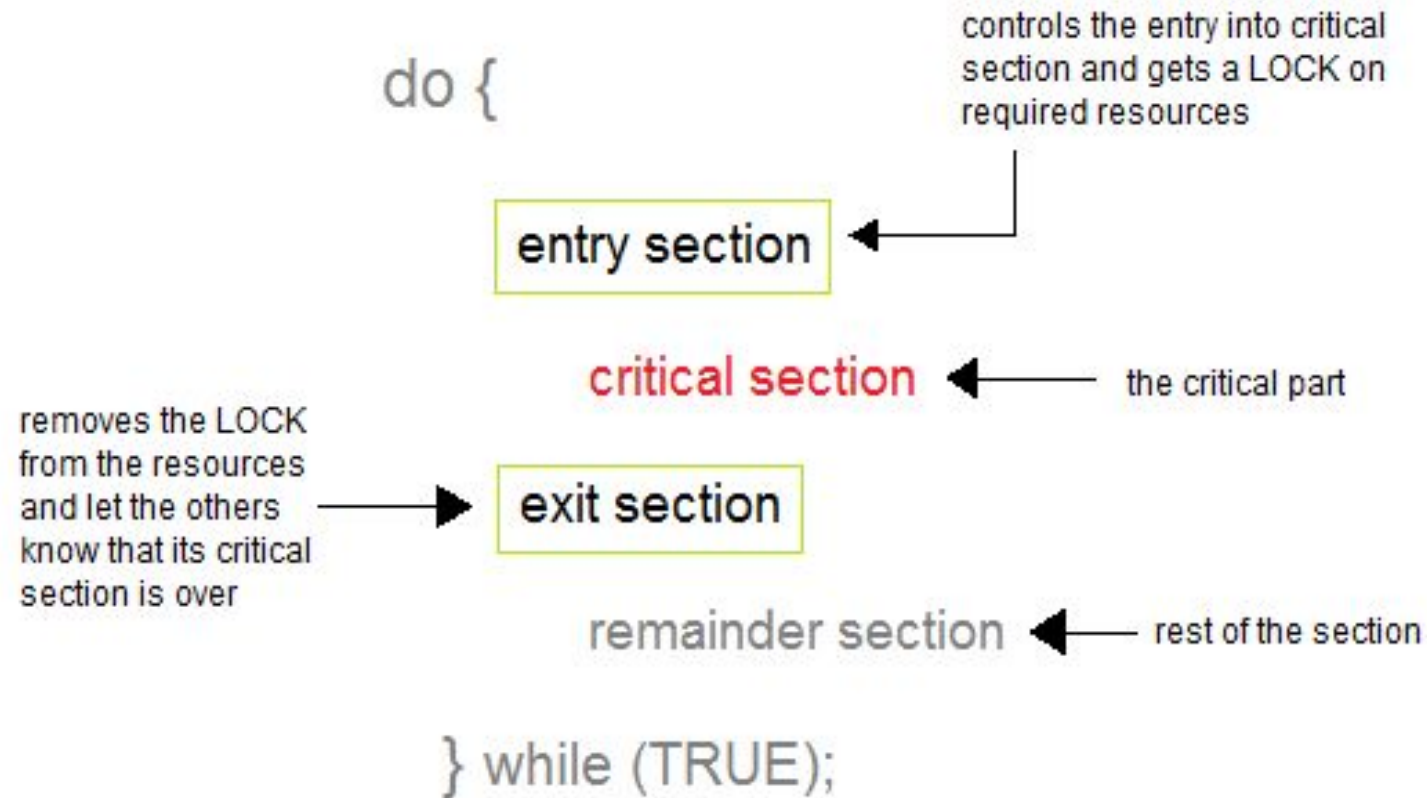
Process Synchronization

- **Process Synchronization** is a technique which is used to coordinate the process that use shared Data.
- Process Synchronization is mainly used for Cooperating Process that shares the resources.
- Process Synchronization was introduced to handle problems that arose while multiple process executions. Some of the problems are discussed below.

1. Critical Section Problem

2. Race Condition

Critical Section Problem:



Solution to Critical Section Problem

- A solution to the critical section problem must satisfy the following three conditions:

1. *Mutual Exclusion*

2. *Progress*

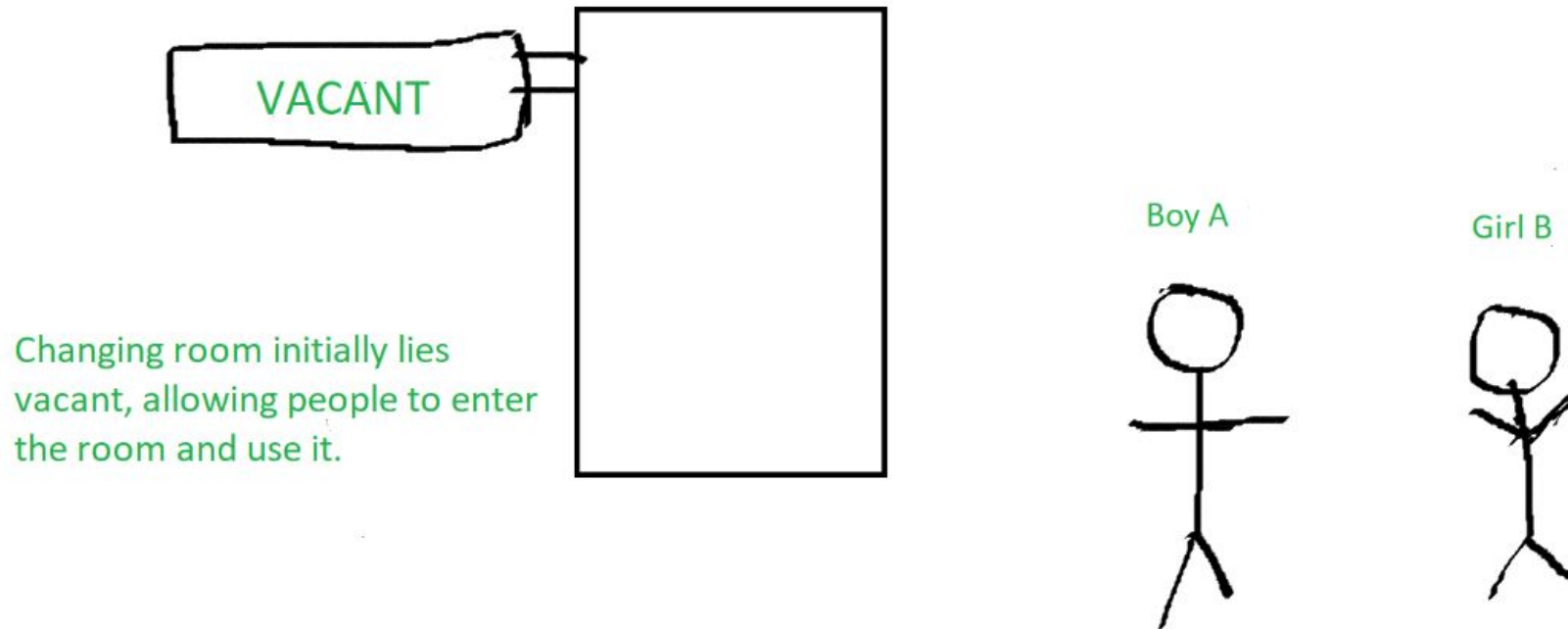
3. *Bounded Waiting*

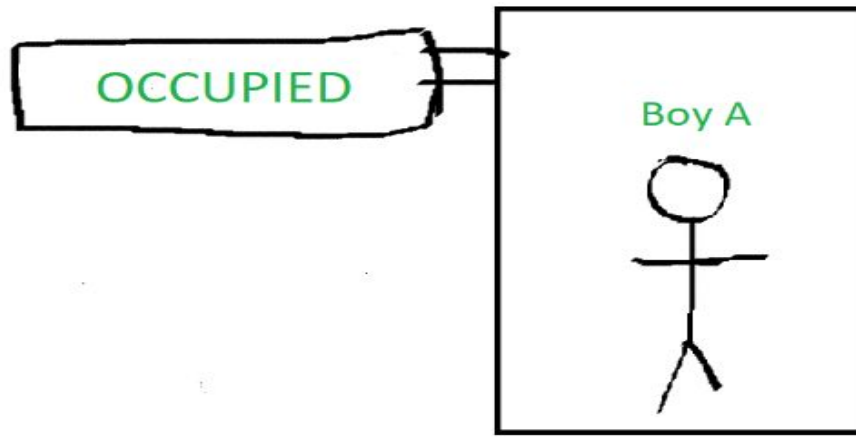
Race Conditions:

- **Two Types of Race Conditions**
 1. Read-modify-write
 2. Check-then-act
- **Preventing Race Conditions: critical section can treat as an atomic instruction.**

Mutual Exclusion in Synchronization

- **Mutual exclusion** is a property of [process synchronization](#) which states that “no two processes can exist in the critical section at any given point of time”.
- **Example:**
In the clothes section of a supermarket, two people are shopping for clothes



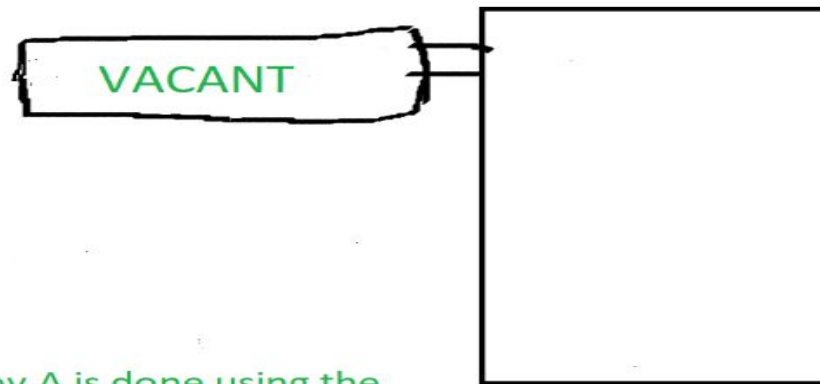


Since Boy A is inside the changing room, the sign on it prevents the others from entering the room.

Girl B



Girl B has to wait outside the changing room till Boy A comes out.



Once Boy A is done using the changing room, he vacates it - making the sign outside the room change.

Since the changing room has been vacated by Boy A, Girl B can enter the changing room.

Girl B



Boy A



Semaphores

- Semaphore was proposed by Dijkstra in 1965 which is a very significant technique to manage concurrent processes by using a simple integer value, which is known as a **semaphore**.
- Semaphores are integer variables that are used to solve the critical section problem by using two atomic operations, wait and signal that are used for process synchronization.
- The definitions of wait and signal are as follows –

-Wait

- The wait operation decrements the value of its argument S, if it is positive. If S is negative or zero, then no operation is performed.

-Signal

- The signal operation increments the value of its argument S.

•Types of Semaphores

1. Counting semaphores
2. Binary semaphores.

•Advantages of Semaphores

•Some of the advantages of semaphores are as follows –

- Semaphores allow only one process into the critical section. They follow the mutual exclusion principle strictly and are much more efficient than some other methods of synchronization.
- There is no resource wastage because of busy waiting in semaphores as processor time is not wasted unnecessarily to check if a condition is fulfilled to allow a process to access the critical section.
- Semaphores are implemented in the machine independent code of the microkernel. So they are machine independent.

•Disadvantages of Semaphores

•Some of the disadvantages of semaphores are as follows –

- Semaphores are complicated so the wait and signal operations must be implemented in the correct order to prevent deadlocks.
- Semaphores are impractical for last scale use as their use leads to loss of modularity. This happens because the wait and signal operations prevent the creation of a structured layout for the system.
- Semaphores may lead to a priority inversion where low priority processes may access the critical section first and high priority processes later.

- **Producer and Consumer problem**
- **Deadlock**
- **THE DINING PHILOSOPHERS PROBLEM**



Thank
YOU